

Gameweek Desk

An availability-verification and editorial review system for a small content operations team. Amaan Javed

Live prototype	PASTE-STREAMLIT-URL-HERE
Video walkthrough	PASTE-VIDEO-URL-HERE

The prototype runs with no API keys configured, on recorded agent verdicts. The orchestration, aggregation and escalation logic are the same code paths used against live APIs; only the two external calls are replayed.

The problem

A six-person FPL advice service with 40,000 subscribers must research injury, suspension and rotation news across roughly 600 players before every gameweek deadline, then decide what to publish. Done manually that is about **25 reviewer-hours per week** of repetitive lookup — most of one person's time — and it caps the product: covering more players or more subscriber segments requires hiring.

This is a stand-in for a shape of problem rather than a claim about any company. The same structure — a deterministic ranking, external research requiring judgement, and a publish decision with real consequences — describes a trust & safety review queue, a refunds desk, or a grant-eligibility screen. Only the vocabulary changes.

Architecture

FPL data → xP model (ridge, cross-validated) → transfer optimiser (shared budget, legal squads) → availability agent (search + LLM, self-consistency) → review queue → **human publish gate**

A **confirmed** availability problem — unanimous across samples, cited, and fresh — removes that player from the candidate pool and the optimiser re-runs. The agent does not merely annotate a recommendation it disagrees with. Anything less certain is escalated to a human with a stated reason.

The human / AI boundary

The AI owns	The human owns
Ingesting and cleaning data; ranking the player pool; searching live sources; reading unstructured news and forming a verdict; drafting the subscriber note; removing confirmed-unavailable players and re-running the optimiser.	Every decision that reaches a subscriber; anything flagged uncertain; the confidence and freshness thresholds; whether a −4 point hit is worth taking.

The system has no write access to any external system. Its only outputs are a draft and a queue. The one action it takes alone — excluding a confirmed-unavailable player — is conservative: the worst case is a missed opportunity, not a false claim published.

Measured results

Metric	Value	What it means
Leaked	2 / 16	Wrong and not escalated. The only outcome that can reach a subscriber. The number that matters.
Caught	3 / 16	Wrong but escalated. The safety net worked.
Over-escalated	2 / 16	Correct but escalated anyway. Pure cost in human attention.
Blocking precision	50%	Half of injury/suspension flags over-read the source. This is why only confirmed flags act unilaterally.
Blocking recall	100%	No genuinely unavailable player was missed on this set.
Cost	\$10.30/GW	600 players, 3 samples each. Manual equivalent ~\$800/GW.

Trade-offs, and which way I called them

- **Escalate more, or leak less.** Every threshold that reduces wrong answers reaching an editor increases the number of correct answers they waste time confirming. There is no setting that minimises both. Exposed as an operator control with the trade-off curve rendered in-app, because the right point differs on a Tuesday and an hour before deadline.
- **Sample the model, not the search.** Self-consistency needs several opinions, but search dominates cost roughly 4:1. The search runs once; the model is sampled three times at varying temperature against the same evidence. This measures instability in the judgement, which is where it was observed.
- **A simple model I can validate over a complex one I cannot.** The earlier version used gradient boosting fit on season-to-date points and predicted on its own training rows — an in-sample restatement presented as a forecast. Replaced with cross-validated ridge regression carrying a residual-derived prediction interval. Lower ceiling, but the output is a forecast with an error bar, which is what a human needs to weigh it.
- **Recorded snapshot as the default.** Reproducible for review, and immune to a third-party outage mid-demo. Live mode hits the real API and degrades to the snapshot rather than erroring.

Judgement under uncertainty: three real failures, three responses

Observed failure	Response
Same player, same input, 60 seconds apart: the agent returned <i>suspended</i> once and <i>injured</i> the next time.	Sampled three times at varying temperature. Disagreement is itself the signal — when samples do not converge the item escalates rather than one verdict being presented confidently.
A verdict of “found guilty of violent conduct, therefore suspended” with no source and no date — a match report read as current squad status.	Every verdict must carry a source URL, publication date and verbatim quote. Uncited blocking claims are inadmissible and escalate.
News from weeks earlier treated as establishing availability today.	Freshness window: evidence beyond it cannot on its own justify a blocking flag.

Known limits

- **Comprehension errors pass every guardrail.** Two eval cases are unanimous, cited, fresh and wrong — a training report mentioning a minor knock read as full fitness, and transfer speculation read as rotation risk. Disagreement, citation and freshness checks cannot catch a misreading. Roughly 1 in 8. This is the residual an editor absorbs, and the reason the review step exists.
- **Search coverage is uneven.** Well-covered players yield good evidence; a squad player at a promoted club may return nothing usable. The system returns UNKNOWN and escalates rather than guessing — but that loads the human exactly where they also know least.
- **Pre-season forecasts are priors, not predictions.** With no minutes played the model leans entirely on last season's rates. The prior weight is displayed in the header rather than buried.
- **Synthetic snapshot inflates model metrics.** Priors were generated from price, so cross-validated R^2 is optimistic. Stated in-app rather than left to be discovered.
- **No provider redundancy.** One search provider, one model provider. Either failing degrades the system to “everything escalates” — safe, but not useful.

What I would build next, in order

- **Close the feedback loop on overrides.** Editor rejections with reasons are labelled data currently being discarded. This turns every week of operation into eval set growth — the highest-value missing piece.
- **Cache verdicts per club, not per player.** Team news is club-level. One search per club rather than per player cuts the dominant cost by roughly an order of magnitude.
- **A second source before any blocking call.** Blocking precision is the weakest measured number; cross-checking against a structured injury feed is the direct fix.
- **Batch overnight, review in the morning.** Latency stops mattering, and a cheaper, slower model becomes viable.

Assumptions are stated explicitly on the Assumptions page in the app. The scenario (team size, subscriber count, manual baseline) is an assumption; the measured results, cost figures and failure modes are computed from the running system. Player club and position data in the snapshot is illustrative and may not reflect current transfers.